How to Make a Simple LLM Call Using LangChain

Making a simple Large Language Model (LLM) call using LangChain is the fundamental building block for building AI-powered applications. LangChain provides a unified interface to interact with various LLM providers like OpenAI, Anthropic, Google, and others, abstracting away the complexities of different APIs and providing consistent methods for invoking language models.

At its core, an LLM call in LangChain involves creating an instance of an LLM class, configuring it with the necessary parameters and API credentials, and then invoking it with a prompt or message. The framework handles the underlying API communication, response parsing, and error handling, allowing developers to focus on the application logic rather than the intricacies of different LLM providers.

LLM in LangChain: An LLM (Large Language Model) in LangChain is an abstraction that represents a language model service. It provides a consistent interface for sending text prompts to various AI models and receiving generated responses, regardless of the underlying provider.

Why LLM Calls Matter

LLM calls form the foundation of modern AI applications, enabling natural language processing, content generation, question answering, and complex reasoning tasks. LangChain's approach to LLM calls provides several advantages over direct API usage. It offers provider abstraction, allowing you to switch between different LLM services without changing your application code. The framework also provides built-in retry logic, error handling, and response parsing, making your applications more robust and reliable.

Furthermore, LangChain's LLM interface integrates seamlessly with other framework components like prompt templates, output parsers, and chains, enabling you to build sophisticated AI workflows. This integration is crucial for creating production-ready applications that require consistent behavior, proper error handling, and the ability to scale across different LLM providers.

Key Benefits: Unified interface across providers, built-in error handling, easy integration with other LangChain components, and simplified provider switching for better cost optimization and performance tuning.

Installation and Setup Requirements

Before making LLM calls, you need to install LangChain and the specific provider packages. The installation process varies depending on which LLM provider you plan to use, but the core LangChain package is always required.

Installing LangChain and Provider Packages

Terminal

```
# Install core LangChain package
pip install langchain

# Install OpenAI integration
pip install langchain-openai

# Install Anthropic integration
pip install langchain-anthropic
```

```
# Install Google integration
pip install langchain-google-genai

# Or install multiple providers at once
pip install langchain langchain-openai langchain-anthropic
```

Provider-Specific Packages: LangChain uses separate packages for different LLM providers to keep dependencies lightweight. Only install the packages for providers you actually plan to use.

Environment Variable Configuration

Most LLM providers require API keys for authentication. The recommended approach is to store these keys as environment variables, which LangChain will automatically detect and use.

Setting Up Environment Variables

Terminal

```
# In your terminal or .env file
export OPENAI_API_KEY="your-openai-api-key-here"
export ANTHROPIC_API_KEY="your-anthropic-api-key-here"
export GOOGLE_API_KEY="your-google-api-key-here"
```

Basic LLM Call Examples

Simple OpenAI LLM Call

Python

```python
from langchain_openai import OpenAI

# Initialize the LLM
llm = OpenAI(
    model_name="gpt-3.5-turbo-instruct",
    temperature=0.7,
    max_tokens=100
)

# Make a simple call
prompt = "What is the capital of France?"
response = llm.invoke(prompt)
print(response)
```

Chat Model Example (Recommended)

Python

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# Initialize the chat model
```

```python
chat_model = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0.7,
    max_tokens=150
)

# Create a message
message = HumanMessage(content="Explain quantum computing in simple terms")

# Make the call
response = chat_model.invoke([message])
print(response.content)
```

Chat Models vs LLMs: Modern applications should use ChatModels (like ChatOpenAI) instead of legacy LLM classes, as they provide better support for conversation history, system messages, and structured interactions.

Anthropic Claude Example

```python
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage

# Initialize Claude
claude = ChatAnthropic(
    model="claude-3-sonnet-20240229",
    temperature=0.5,
    max_tokens=200
)

# Make a call
message = HumanMessage(content="Write a haiku about programming")
response = claude.invoke([message])
print(response.content)
```

Configuration Parameters

LLM calls can be customized with various parameters that control the model's behavior. Understanding these parameters is crucial for getting the desired output from your language model.

Key Parameters:

- **Temperature:** Controls randomness (0.0 = deterministic, 1.0 = very random)
- **Max Tokens:** Maximum number of tokens in the response
- **Model:** Specific model variant to use
- **Top P:** Nucleus sampling parameter for controlling diversity

Advanced Configuration Example

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

# Create a highly configured chat model
llm = ChatOpenAI(
    model="gpt-4",
    temperature=0.3,        # Lower temperature for more focused responses
    max_tokens=500,         # Longer responses allowed
    top_p=0.9,             # Nucleus sampling
    frequency_penalty=0.0, # Avoid repeated phrases
    presence_penalty=0.0   # Encourage new topics
)

# Use system and human messages
messages = [
    SystemMessage(content="You are a helpful Python programming tutor."),
    HumanMessage(content="Explain list comprehensions with examples")
]

response = llm.invoke(messages)
print(response.content)
```

Error Handling and Best Practices

Robust LLM applications require proper error handling to manage API failures, rate limits, and network issues. LangChain provides built-in retry mechanisms, but you should also implement your own error handling strategies.

Error Handling Example

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
import time

def make_llm_call_with_retry(prompt, max_retries=3):
    llm = ChatOpenAI(
        model="gpt-3.5-turbo",
        temperature=0.7,
        request_timeout=30  # Set timeout
    )

    for attempt in range(max_retries):
        try:
            message = HumanMessage(content=prompt)
            response = llm.invoke([message])
            return response.content

        except Exception as e:
            print(f"Attempt {attempt + 1} failed: {e}")
            if attempt < max_retries - 1:
                time.sleep(2 ** attempt)  # Exponential backoff
            else:
                raise e

# Usage
try:
    result = make_llm_call_with_retry("What is machine learning?")
    print(result)
except Exception as e:
```

```
    print(f"Failed to get response: {e}")
```

Asynchronous LLM Calls

```python
import asyncio
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

async def async_llm_call(prompt):
    llm = ChatOpenAI(
        model="gpt-3.5-turbo",
        temperature=0.7
    )

    message = HumanMessage(content=prompt)
    response = await llm.ainvoke([message])
    return response.content

async def main():
    # Process multiple prompts concurrently
    prompts = [
        "What is Python?",
        "Explain machine learning",
        "What is cloud computing?"
    ]

    tasks = [async_llm_call(prompt) for prompt in prompts]
    responses = await asyncio.gather(*tasks)

    for prompt, response in zip(prompts, responses):
        print(f"Q: {prompt}")
        print(f"A: {response}\n")

# Run the async function
asyncio.run(main())
```

Best Practices

Following best practices ensures your LLM calls are efficient, reliable, and cost-effective. These practices have been developed through extensive use of LangChain in production environments.

- Always store API keys in environment variables, never hardcode them in your source code
- Use appropriate temperature settings - lower for factual tasks, higher for creative tasks
- Set reasonable token limits to control costs and response length
- Implement proper error handling and retry logic for production applications
- Use chat models instead of legacy LLM classes for modern applications
- Consider using async calls for better performance when processing multiple requests
- Monitor your API usage and costs, especially in production environments
- Cache responses when appropriate to reduce redundant API calls

> **Cost Optimization:** Different models have different pricing structures. Start with smaller, faster models for development and testing, then scale to more powerful models for production when needed.

Common Mistakes

- Forgetting to install provider-specific packages, leading to import errors
- Not setting API keys properly, causing authentication failures
- Using synchronous calls in applications that need to handle multiple requests concurrently
- Not implementing proper error handling, making applications fragile
- Setting temperature too high for factual queries, resulting in inconsistent responses
- Not monitoring token usage, leading to unexpected costs
- Hardcoding API keys in source code, creating security vulnerabilities
- Using deprecated LLM classes instead of modern ChatModel interfaces

Complete Working Example

Python

```python
import os
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

# Ensure API key is set
if not os.getenv("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY environment variable not set")

def create_llm_assistant():
    """Create a configured LLM assistant"""
    return ChatOpenAI(
        model="gpt-3.5-turbo",
        temperature=0.7,
        max_tokens=300,
        request_timeout=30
    )

def ask_question(question, context=None):
    """Ask a question to the LLM with optional context"""
    llm = create_llm_assistant()

    messages = []

    if context:
        messages.append(SystemMessage(content=context))

    messages.append(HumanMessage(content=question))

    try:
        response = llm.invoke(messages)
        return response.content
    except Exception as e:
        return f"Error: {str(e)}"

# Example usage
if __name__ == "__main__":
    # Simple question
    answer1 = ask_question("What is the capital of Japan?")
    print(f"Answer 1: {answer1}")

    # Question with context
    context = "You are a helpful coding tutor specializing in Python."
    answer2 = ask_question(
        "How do I create a list in Python?",
```

```
        context=context
    )
    print(f"Answer 2: {answer2}")
```

Performance Impact

LLM calls have significant performance implications that must be considered in application design. The latency of LLM calls can range from hundreds of milliseconds to several seconds, depending on the model size, complexity of the prompt, and current API load. Token usage directly impacts both response time and costs, making efficient prompt design crucial for production applications.

When building applications that require multiple LLM calls, consider using asynchronous processing to improve overall throughput. Caching strategies can also dramatically improve performance for repeated or similar queries. Additionally, choosing the right model for your use case balances performance needs with cost considerations - smaller models like GPT-3.5 are faster and cheaper, while larger models like GPT-4 provide better quality at the cost of increased latency and expense.

Performance Tips: Use streaming for long responses, implement caching for repeated queries, choose appropriate model sizes for your needs, and consider using multiple smaller calls instead of one large complex call when possible.

Advanced Patterns and Techniques

As you become more comfortable with basic LLM calls, several advanced patterns can help you build more sophisticated applications. These patterns address common challenges in production environments and provide frameworks for handling complex use cases.

Streaming Responses

Python

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

def stream_response(prompt):
    llm = ChatOpenAI(
        model="gpt-3.5-turbo",
        temperature=0.7,
        streaming=True  # Enable streaming
    )

    message = HumanMessage(content=prompt)

    # Stream the response token by token
    for chunk in llm.stream([message]):
        print(chunk.content, end="", flush=True)

    print()  # New line after complete response

# Usage for real-time response display
stream_response("Write a short story about a robot learning to paint")
```

Batch Processing Multiple Prompts

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

def batch_process_prompts(prompts, batch_size=5):
    llm = ChatOpenAI(
        model="gpt-3.5-turbo",
        temperature=0.7
    )

    results = []

    # Process prompts in batches to manage rate limits
    for i in range(0, len(prompts), batch_size):
        batch = prompts[i:i + batch_size]
        batch_messages = [[HumanMessage(content=prompt)] for prompt in batch]

        # Use batch method for efficiency
        batch_responses = llm.batch(batch_messages)

        for response in batch_responses:
            results.append(response.content)

    return results

# Example usage
prompts = [
    "Summarize the benefits of renewable energy",
    "Explain the water cycle",
    "What is photosynthesis?",
    "Describe the solar system"
]

responses = batch_process_prompts(prompts)
for prompt, response in zip(prompts, responses):
    print(f"Q: {prompt}")
    print(f"A: {response[:100]}...\n")
```

Response Caching for Performance

```python
import hashlib
import json
import os
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

class CachedLLM:
    def __init__(self, cache_dir="llm_cache"):
        self.llm = ChatOpenAI(
            model="gpt-3.5-turbo",
            temperature=0.7
        )
        self.cache_dir = cache_dir
        os.makedirs(cache_dir, exist_ok=True)

    def _get_cache_key(self, prompt):
        # Create unique hash for the prompt
        return hashlib.md5(prompt.encode()).hexdigest()
```

```python
    def _get_cache_path(self, cache_key):
        return os.path.join(self.cache_dir, f"{cache_key}.json")

    def invoke(self, prompt):
        cache_key = self._get_cache_key(prompt)
        cache_path = self._get_cache_path(cache_key)

        # Check if response is cached
        if os.path.exists(cache_path):
            with open(cache_path, 'r') as f:
                cached_response = json.load(f)
                print("Cache hit!")
                return cached_response['response']

        # Make LLM call if not cached
        print("Cache miss, calling LLM...")
        message = HumanMessage(content=prompt)
        response = self.llm.invoke([message])

        # Cache the response
        cache_data = {
            'prompt': prompt,
            'response': response.content
        }

        with open(cache_path, 'w') as f:
            json.dump(cache_data, f)

        return response.content

# Usage example
cached_llm = CachedLLM()

# First call - will hit the API
result1 = cached_llm.invoke("What is machine learning?")
print(result1[:100])

# Second call - will use cache
result2 = cached_llm.invoke("What is machine learning?")
print(result2[:100])
```

Integration with Other LangChain Components

LLM calls become much more powerful when combined with other LangChain components. This integration allows you to build complex workflows that can handle structured data, maintain conversation context, and perform multi-step reasoning tasks.

Using LLMs with Prompt Templates

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Create a prompt template
prompt_template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful {role} with expertise in {domain}."),
    ("human", "Please explain {topic} in simple terms.")
])

# Initialize LLM and output parser
```

```python
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
output_parser = StrOutputParser()

# Create a chain
chain = prompt_template | llm | output_parser

# Use the chain with different inputs
result = chain.invoke({
    "role": "science teacher",
    "domain": "physics",
    "topic": "quantum mechanics"
})

print(result)
```

Memory-Enabled Conversations

```python
from langchain_openai import ChatOpenAI
from langchain.memory import ConversationBufferMemory
from langchain_core.messages import HumanMessage, AIMessage

class ConversationalAssistant:
    def __init__(self):
        self.llm = ChatOpenAI(
            model="gpt-3.5-turbo",
            temperature=0.7
        )
        self.conversation_history = []

    def chat(self, user_input):
        # Add user message to history
        self.conversation_history.append(HumanMessage(content=user_input))

        # Get response from LLM
        response = self.llm.invoke(self.conversation_history)

        # Add AI response to history
        self.conversation_history.append(AIMessage(content=response.content))

        return response.content

    def clear_history(self):
        self.conversation_history = []

# Usage example
assistant = ConversationalAssistant()

print("Assistant: Hello! How can I help you today?")

# Simulate a conversation
response1 = assistant.chat("My name is Alice and I'm learning Python.")
print(f"Assistant: {response1}")

response2 = assistant.chat("Can you recommend some Python books for beginners?")
print(f"Assistant: {response2}")

response3 = assistant.chat("Remember my name and suggest a project suitable for my
level.")
print(f"Assistant: {response3}")
```

Testing and Debugging LLM Calls

Testing LLM applications presents unique challenges due to the non-deterministic nature of language models. However, there are several strategies you can employ to ensure your LLM calls behave consistently and reliably.

Testing LLM Applications

```python
import unittest
from unittest.mock import Mock, patch
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage

class TestLLMCalls(unittest.TestCase):

    def setUp(self):
        self.llm = ChatOpenAI(
            model="gpt-3.5-turbo",
            temperature=0  # Use 0 temperature for consistent results
        )

    @patch('langchain_openai.ChatOpenAI.invoke')
    def test_llm_call_with_mock(self, mock_invoke):
        # Mock the LLM response
        mock_response = AIMessage(content="Paris is the capital of France.")
        mock_invoke.return_value = mock_response

        # Test the call
        message = HumanMessage(content="What is the capital of France?")
        result = self.llm.invoke([message])

        # Assertions
        self.assertEqual(result.content, "Paris is the capital of France.")
        mock_invoke.assert_called_once_with([message])

    def test_llm_response_structure(self):
        # Test with actual LLM call (integration test)
        message = HumanMessage(content="Say 'Hello World'")
        response = self.llm.invoke([message])

        # Test response structure
        self.assertIsInstance(response, AIMessage)
        self.assertIsInstance(response.content, str)
        self.assertGreater(len(response.content), 0)

    def test_llm_error_handling(self):
        # Test with invalid input
        with self.assertRaises(Exception):
            # This should raise an exception
            invalid_llm = ChatOpenAI(api_key="invalid_key")
            message = HumanMessage(content="Test")
            invalid_llm.invoke([message])

if __name__ == '__main__':
    unittest.main()
```

Testing Strategies: Use low temperature settings for consistent outputs, mock LLM responses for unit tests, test response structure and format, and implement integration tests with real API calls for critical paths.

Production Considerations

Deploying LLM applications to production requires careful consideration of several factors including cost management, rate limiting, security, and monitoring. These considerations become critical as your application scales and serves real users.

Production Checklist:

- Implement proper logging and monitoring for API calls and costs
- Set up rate limiting to prevent API quota exhaustion
- Use environment-specific configurations for different deployment stages
- Implement circuit breakers for handling API failures gracefully
- Monitor token usage and costs continuously
- Set up alerts for unusual usage patterns or errors

Production-Ready LLM Service

Python

```python
import os
import logging
import time
from functools import wraps
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def rate_limit(calls_per_minute=60):
    """Decorator to implement rate limiting"""
    min_interval = 60.0 / calls_per_minute
    last_called = [0.0]

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_called[0]
            left_to_wait = min_interval - elapsed
            if left_to_wait > 0:
                time.sleep(left_to_wait)
            ret = func(*args, **kwargs)
            last_called[0] = time.time()
            return ret
        return wrapper
    return decorator

class ProductionLLMService:
    def __init__(self):
        self.llm = ChatOpenAI(
            model=os.getenv("LLM_MODEL", "gpt-3.5-turbo"),
            temperature=float(os.getenv("LLM_TEMPERATURE", "0.7")),
            max_tokens=int(os.getenv("LLM_MAX_TOKENS", "500")),
            request_timeout=int(os.getenv("LLM_TIMEOUT", "30"))
        )
        self.total_tokens_used = 0
        self.total_calls = 0

    @rate_limit(calls_per_minute=50)
```

```python
    def generate_response(self, prompt, user_id=None):
        """Generate response with logging and monitoring"""
        start_time = time.time()

        try:
            logger.info(f"Processing request for user: {user_id}")

            message = HumanMessage(content=prompt)
            response = self.llm.invoke([message])

            # Update metrics
            self.total_calls += 1
            response_time = time.time() - start_time

            logger.info(f"Request completed in {response_time:.2f}s")

            return {
                "response": response.content,
                "success": True,
                "response_time": response_time
            }

        except Exception as e:
            error_time = time.time() - start_time
            logger.error(f"Request failed after {error_time:.2f}s: {str(e)}")

            return {
                "response": "I apologize, but I'm unable to process your request right
now.",
                "success": False,
                "error": str(e),
                "response_time": error_time
            }

    def get_metrics(self):
        """Return service metrics"""
        return {
            "total_calls": self.total_calls,
            "total_tokens_used": self.total_tokens_used
        }

# Usage example
service = ProductionLLMService()
result = service.generate_response(
    "What is artificial intelligence?",
    user_id="user123"
)
print(result)
```

Key Takeaway: Making simple LLM calls with LangChain is straightforward, but building production-ready applications requires careful consideration of error handling, performance optimization, cost management, and proper integration patterns. Start simple and gradually add complexity as your needs grow.

For more educational content and insights -

Connect with Sumedh